

When Should an LLM Search?

A Calibration-Centric Evaluation of Search Behavior and a Hidden Generalization Gap Under Natural Paraphrases

Anonymous Authors

Paper under double-blind review at ICLR 2026

Abstract

Modern LLMs are increasingly deployed with web-search tools, yet we lack a principled methodology for asking the most basic question about this behavior: *given that a model has the option to search, does it search when it should, and skip when it shouldn't?* Existing evaluations measure end-task accuracy or whether-or-not to use a tool at the question level; they do not localize search decisions to specific reasoning steps, nor do they reference the model's own internal knowledge as the natural decision boundary. We close this gap with an evaluation protocol on 600 staleness-filtered MuSiQue (Trivedi et al., 2022) multi-hop questions. We first estimate per-hop *parametric uncertainty* via semantic entropy (Farquhar et al., 2024) over 5 closed-book samples, then run the same model on the full question with web search enabled, and use an LLM judge to attribute each emitted search query back to a specific hop. The cross-product of “certain vs. uncertain” and “searched vs. not searched” — refined with a sequential-redundancy flag and a cross-hop-coverage flag — partitions every per-hop decision into six interpretable behavioral categories: three correct decisions (*Necessary search*, *Correct skip*, *Cross-hop covered*) and three search errors (*Truly missed*, *Sequentially redundant*, *Parametrically redundant*). Applied to Gemini 3.1 Pro, Qwen 3.5 (122B) and Nemotron Nano (30B), the extended profile exposes three sharply different failure styles invisible to aggregate metrics. We then show the central calibration finding: the AUROC of entropy as a predictor of the search decision sits in the 0.56–0.62 range across all three models — search is barely aligned with the model's own uncertainty. Finally, we introduce a paraphrase probe: every question is rewritten into a natural, user-styled question that preserves the underlying information need but removes the benchmark's explicit hop chain. All three models search substantially less under the natural rewrite, and the searches they retain are re-allocated *away from* the uncertain hops the same model had identified on the benchmark. The dominant new category is *Truly missed* — a hop the model is internally uncertain about and never queries the web for — exposing a hidden generalization gap that existing benchmarks do not measure.

1 Introduction

Retrieval-augmented LLMs are a default deployment pattern (Huang and Huang, 2024). The standard evaluation question is whether retrieval improves end-task scores; a far less studied question is whether the model uses retrieval *appropriately* — i.e. whether it triggers searches on facts it does not know and skips them on facts it does. Tool-use benchmarks such as WTU-Eval (Ning et al., 2024) frame the problem at the granularity of the entire question (use the tool or not), and the broader RAG / tug-of-war literature (Cheng et al., 2024; Wu et al., 2024a; Tao et al., 2024; Kim et al., 2025; Farahani and Johansson, 2024) focuses on how models *integrate* external evidence once retrieved. To our knowledge, no prior work isolates and evaluates the *per-step* decision to invoke search, conditioned on the model's own internal knowledge of that step.

This is the question we address. Crucially, we deliberately do *not* ground our analysis in end-task correctness. The premise of the paper is that an external retrieval decision should be aligned with the model's own internal epistemic state — regardless of whether the model would, downstream, happen to produce

the right final answer. A model that searches questions it knows perfectly and skips questions it doesn't is *mis-calibrated* as a search agent, independently of whether retrieval rescues the final answer.

Contributions.

1. An **evaluation protocol** that pairs per-hop semantic entropy (from closed-book sampling) with per-hop search attribution (LLM judge) on multi-hop questions (§3).
2. A **six-category extended search-calibration profile** that partitions every per-hop decision into three correct cells and three distinct error cells; the correct cells distinguish *Cross-hop covered* (uncertain, not directly searched but answered as a by-product of another hop's search) from *Truly missed* (uncertain, never searched and never covered) on the error side, and the error cells separate *Sequentially redundant* (re-searching the same already-known content within a trace) from *Parametrically redundant* (searching content the model already knows parametrically) (§4).
3. A **cross-model calibration analysis** on three production-style LLMs showing (i) the six-category profile yields three sharply different failure archetypes, and (ii) the AUROC of entropy $\rightarrow$ search sits near chance for all three, the headline diagnostic of mis-calibration (§4,5).
4. A **natural-paraphrase generalization probe** showing that a cosmetic rephrasing that preserves the information need causes all three models to search less — and, more strikingly, to redirect their remaining searches *away from* their own uncertain hops, with *Truly missed* as the dominant new error cell (§6).

We argue that the paraphrase result is the most important finding of the paper. It is consistent with the literature on compositional brittleness (Li et al., 2024; Yang et al., 2024; Wu et al., 2024b) but reframes it as a *behavioral* gap, not a reasoning one: even when the underlying parametric knowledge has not changed, a cosmetic paraphrase causes the model to mis-allocate its search budget.

2 Related Work

Tool-use and search-decision evaluation. WTU-Eval (Ning et al., 2024) is the closest prior benchmark, measuring whether models correctly decide to use a tool at the *question level*. Self-RAG (Asai et al., 2024) and Toolformer (Schick et al., 2023) train models to insert retrieval/tool calls during generation but evaluate downstream task accuracy rather than per-decision calibration. RARE (Wang et al., 2025) explicitly models when to retrieve as a reasoning step, but again evaluates with task metrics. We are not aware of work that (i) localizes search decisions to specific sub-problems within a multi-step question and (ii) grounds the evaluation in the model's own internal uncertainty about those sub-problems.

Parametric vs. contextual knowledge. A growing line studies how LLMs balance internal priors against retrieved context (Cheng et al., 2024; Wu et al., 2024a; Farahani and Johansson, 2024; Kim et al., 2025; Wadhwa et al., 2026; Tao et al., 2024; Zhang et al., 2024). These works probe *conflict resolution* (what happens when the two disagree) rather than the upstream question of *when to invoke retrieval at all*. We treat their findings as motivation: if context can override correct parametric knowledge, then unnecessary searches are not free, and the per-step decision matters.

Uncertainty quantification. We use semantic entropy (Farquhar et al., 2024; Kunitomo-Jacquin et al., 2026) as our operational definition of “knows / does not know,” with semantic clustering by an LLM judge over multiple closed-book samples. This is a deliberate choice: semantic entropy correlates well with hallucination (Farquhar et al., 2024) and is cheaper than fine-grained atomic factuality (Min et al., 2023).

Multi-hop QA and compositional reasoning. MuSiQue (Trivedi et al., 2022) supplies the hop-decomposed questions we need; MRKE (Wu et al., 2024b) and the compositional-reasoning literature (Li et al., 2024; Yang et al., 2024) document the compositional brittleness that makes search a natural intervention. We add: search *behavior* itself is brittle to surface form. For trace-level evaluation methodology we follow the survey of Lee and Hockenmaier (2025).

3 Protocol

We evaluate a model M on a dataset of multi-hop questions $Q = \{q_i\}$, where each q_i decomposes into an ordered chain of sub-questions (hops) $h_{i,1}, \dots, h_{i,k_i}$ with gold answers $a_{i,j}$. The protocol has four stages.

Stage 0 — Staleness filtering. MuSiQue contains time-sensitive questions whose gold answers may have changed since the dataset’s release (e.g. office-holders, populations). Such questions confound a search-behavior study because the correct action — search to refresh — depends on external state, not on internal knowledge. We classify each question with a `gpt-oss:20b` LLM judge that flags whether the gold answer is plausibly time-dependent. From an initial pool we retain **600 non-stale questions** as our evaluation set.

Stage 1 — Per-hop parametric uncertainty. For each example q_i and each hop $h_{i,j}$, we sample the model $N = 5$ times *without search access*, recording the free-text answer. A separate clustering agent (`gpt-oss:120b`) groups the 5 answers into semantic equivalence classes, and we compute *semantic entropy* in bits over the cluster distribution. Because $N = 5$ admits only seven partition signatures, semantic entropy takes one of seven discrete values:

$$H \in \{0, 0.722, 0.971, 1.371, 1.522, 1.922, 2.322\} \text{ bits,}$$

corresponding to partitions $[5], [4,1], [3,2], [3,1,1], [2,2,1], [2,1,1,1], [1,1,1,1,1]$. We say a hop is **certain** if its semantic entropy is 0 and the closed-book sampling cluster is the gold-equivalent one. We say it is **uncertain** otherwise. The conjunction is important: entropy-only certainty would admit the failure mode of confident-wrong, where the model returns the same incorrect answer 5 times.

Stage 2 — Full-question, search-augmented run. We prompt the model with the *aggregate* multi-hop question and a web-search tool (Brave Search), allowing it to interleave search calls and reasoning freely. We record the full trace (every assistant turn, tool call, tool response) and the final answer.

Stage 3 — Query-to-hop attribution. The aggregate trace contains a sequence of search queries $\{Q_1, \dots, Q_T\}$. We use `gpt-oss:120b` as an LLM judge to attribute each query to one of the k_i hops, using the hop questions, the query string, and the preceding reasoning as context. The judge returns a confidence label and may flag a query as “aggregate / general” if it does not cleanly serve one hop. After attribution, each hop carries a binary $\text{searched}_{i,j} \in \{0,1\}$ and a count $\text{queries_assigned}_{i,j}$. Two trace-level annotations enrich the binary search flag:

- **seq_redundant** — the search for this hop is sequentially redundant: an earlier query in the same trace already returned the same content.
- **missed_search_answer_found_cross_hop** — although no query was directly attributed to this hop, a query attributed to a different hop returned the gold-equivalent content for this hop (the hop was *covered* as a by-product).

Models. We evaluate **Gemini 3.1 Pro** (proprietary, large), **Qwen 3.5 122B** (open-weight, large), and **Nemotron Nano 30B** (open-weight, mid-size). All three are accessed via their native APIs; non-Gemini models are served via Ollama. We do not fine-tune; this is a zero-shot evaluation of pretrained behavior.

4 Extended Search-Calibration Profile per Model

Cross-tabulating the per-hop annotations from Stages 1–3 partitions every per-hop decision into one of six interpretable behavioral categories (Table 1). Three are correct decisions; three are search errors with distinct semantics.

Figure 1 compresses the entire main result of the paper into a single chart: the four-cell (E / PR / M / CP) collapse of the six-category profile, shown per model and per dataset side by side. Reading left-to-right, the first three bars are the three models on benchmark MuSiQue; the next three are the same models on the

Table 1: The six extended search-calibration categories. The first three are correct decisions; the remaining three are distinct error modes. “Searched” refers to whether the LLM-judge attributed at least one search query to the hop. **seq_red** = sequentially redundant within the trace; **x-hop** = the gold-equivalent content was returned by a query attributed to a different hop.

Category	Certain?	Searched?	seq_red	x-hop	Reading
Necessary search	no	yes	—	—	Correct: search was warranted
Correct skip	yes	no	—	—	Correct: model knew, did not waste a call
Cross-hop covered	no	no	—	yes	Correct: hop recovered via another hop’s query
Truly missed	no	no	—	no	Error: gap left open
Seq. redundant	yes	yes	yes	—	Error: re-querying already-known content
Par. redundant	yes	yes	no	—	Error: searching a hop the model knew

natural paraphrase. The visual jumps that the rest of the paper unpacks are already visible here: the red (Missed) segment grows substantially in every natural-rewrite bar, the green (Effective) segment shrinks correspondingly, and the orange (Parametrically Redundant) segment contracts as overall search volume falls — without a compensating gain in coverage.

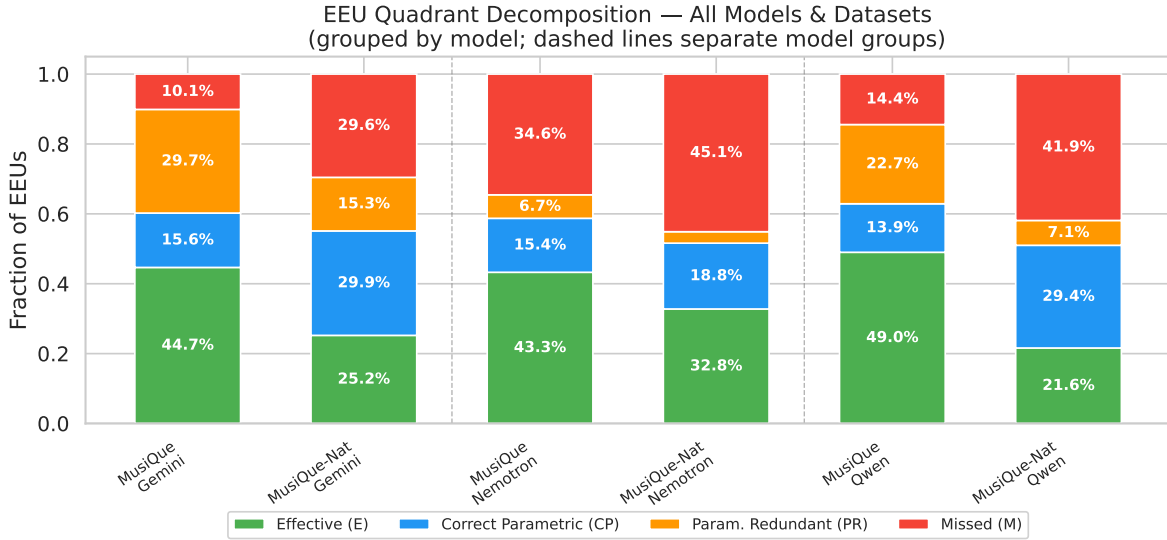


Figure 1: **Headline figure.** Four-cell taxonomy collapse of the six-category profile (E = Effective / Necessary search, CP = Correct Parametric / Correct skip, PR = Parametrically Redundant, M = Missed) per model and per dataset. Each bar sums to 100% of that (model, dataset)’s 1800 hops. Across all three models, the natural-paraphrase bars show a sharp expansion of the red (Missed) segment and a contraction of the green (Effective) segment — the central result of the paper.

Table 2 gives the full six-category breakdown per model on the benchmark. The extended categories distinguish two flavours of redundancy (sequential vs. parametric) and isolate the Cross-hop covered category, which would otherwise be conflated with Missed even though the hop’s information need is in fact satisfied.

Figure 2 shows the full extended-calibration plot for one representative model (Gemini 3.1 Pro), with the benchmark and natural-paraphrase bars overlaid per category. The remaining two per-model plots are in Appendix A; they tell the same story with different baseline distributions, as Table 4 in §6 quantifies.

Three model archetypes. The category mix reveals qualitatively different calibration failures:

- **Gemini** is a *high-volume, redundancy-prone* searcher. Its dominant error mode is *Parametrically redundant* (20.5%) plus *Sequentially redundant* (9.2%) — nearly 30% of its hops are wasted searches on content it

Table 2: Extended search-calibration profile per model on MuSiQue (% of 1800 hops). Left of the rule: correct decisions. Right: distinct error modes.

Model	Necessary search	Correct Correct skip	Cross-hop covered	Truly missed	Errors Seq. redundant	Par. redundant
Gemini 3.1 Pro	44.7	15.6	5.1	5.1	9.2	20.5
Nemotron Nano 30B	43.3	15.4	10.2	24.3	1.3	5.4
Qwen 3.5 122B	49.0	13.9	7.7	6.7	2.8	19.8

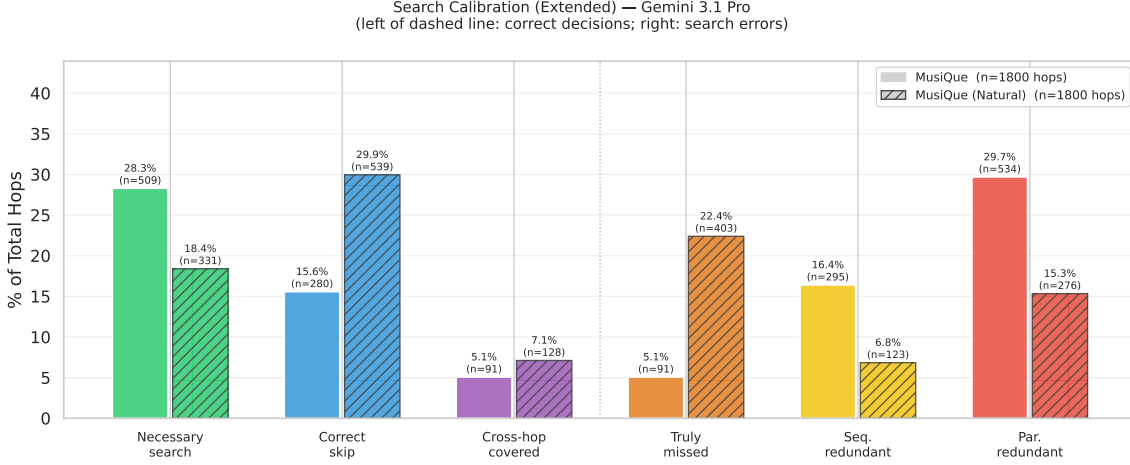


Figure 2: Extended search-calibration profile for Gemini 3.1 Pro. Solid bars: benchmark MuSiQue; hatched bars: natural paraphrase. Left of the dashed line are correct decisions (Necessary search, Correct skip, Cross-hop covered); right of the dashed line are the three error categories. The benchmark→natural shift moves mass from Necessary search into Truly missed (and from Par. redundant into Correct skip), exactly the calibration regression Table 4 reports. Plots for the other two models are in Figure 4.

already has, either in its parameters or in earlier turns of the trace. Coverage of uncertain hops, however, is excellent: *Truly missed* is only 5.1%.

- **Nemotron** is the inverse archetype — a *coverage-failing under-searcher*. Redundancy of both kinds is small (1.3% + 5.4%), but *Truly missed* dominates at 24.3%: nearly a quarter of all hops are uncertain and the model never even attempts a query.
- **Qwen** sits between the two. It searches more uncertain hops than the other two (49.0% *Necessary search*) but pays for it with 19.8% *Par. redundant*, mirroring Gemini’s redundancy pattern at slightly lower volume.

The three-way error decomposition is essential here. Any binary collapse to “correct search-decision” versus “error” would hide that Gemini and Nemotron fail in opposite ways. Cross-hop coverage — the case where an uncertain hop is recovered as a by-product of another hop’s query — is non-negligible (5.1–10.2%) for all three; we count it as a correct outcome, since the hop’s information need is in fact satisfied, but it is invisible in a strict “did the model search this hop” framing and would be misread as a miss without this annotation.

Cross-model statistical contrast. A Kruskal–Wallis test on the per-hop entropy distributions rejects the null of equal distributions across the three models ($H = 499.2$, $p < 10^{-108}$), confirming that the behavioral differences are not driven by trivially different uncertainty inputs. A Mann–Whitney U test on the entropy of *searched* hops shows Nemotron searches substantially more uncertain hops on average than Gemini ($p < 10^{-97}$), consistent with the very different positions of the two models on the redundancy/coverage axis.

5 The Headline Calibration Finding: Entropy Barely Predicts Search

If models routed search by their own uncertainty, the AUROC of entropy $\rightarrow$ search would approach 1.0. We observe instead AUROC in the 0.56–0.62 range across all three models (Figure 3). This is the central diagnostic of the paper: *the latent uncertainty signal exists — but it is not the dominant driver of when these models decide to search*.

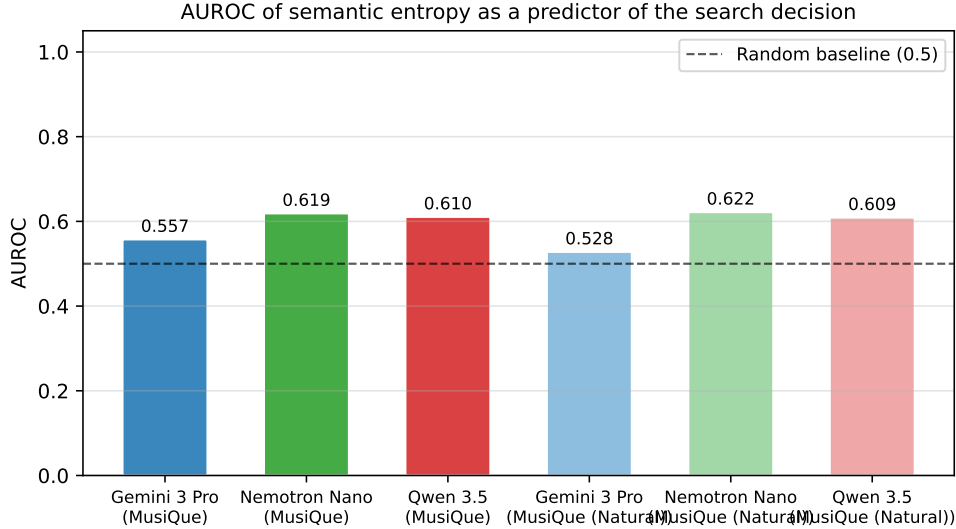


Figure 3: AUROC of semantic entropy as a predictor of the per-hop search decision, per (model, dataset). The dashed line marks the random baseline (AUROC = 0.5). All six combinations fall in the 0.53–0.62 range; for Gemini under natural paraphrasing, AUROC drops to 0.528 — effectively chance.

Two observations sharpen this finding. (1) Even the *best* of the three, Nemotron, clears AUROC 0.6 only modestly — and pays for that calibration with the highest *Truly missed* rate of the three (Table 2). (2) The qualitative shape of the per-entropy-level search rate is roughly flat for all three models rather than monotone — search is approximately *binary* (the model either searches a hop or doesn’t, with little graded response to the magnitude of internal uncertainty). The per-entropy-level rate plot supporting this binary-behavior claim is in Appendix B; the qualitative pattern it shows is consistent with the near-chance AUROC reported here.

6 Generalization Under Natural Paraphrases

The protocol so far evaluates models on benchmark questions, which in MuSiQue expose the hop chain syntactically (“What administrative territorial entity includes the place where X was educated?”). A natural user with the same information need would phrase the question differently (“I’m curious about X’s background — what region of the country did he study in?”). Two facts about this distinction motivate our second experiment: (i) the underlying information need is identical (same gold answer, same hop chain semantically); (ii) the surface form differs (no nested relative clauses, no exposed bridge entities, often a request for a short narrative rather than a single fact).

If the search-decision policy were governed by the model’s internal uncertainty about facts, behavior should be approximately invariant to this surface change. We test this directly.

Paraphrase generation. We use `gpt-oss:20b` to paraphrase each of the 600 MuSiQue questions into a natural, goal-oriented rewrite. The system prompt forbids exposing intermediate bridge entities, forbids the “X of Y where Z did W” benchmark pattern, and asks for phrasing that invites 2–4 explanatory sentences. We retain the same gold answers and the same hop decompositions for downstream attribution. We then

re-run only Stages 2–3 of the protocol (the search-augmented run and hop attribution); Stage 1 parametric uncertainty is *reused from the original questions* because the hop chain is unchanged.

Result 1 — Search volume collapses. All three models cut search volume substantially (Table 3): Qwen by 4.2×, Gemini by 3.1×, Nemotron by 1.8×. The information need is identical; only the phrasing changed.

Table 3: Search volume per question, benchmark vs. natural paraphrase (paired across 600 items). Wilcoxon signed-rank test, two-sided.

Model	Benchmark (mean)	Natural (mean)	Ratio	Wilcoxon p
Gemini 3.1 Pro	15.20	4.93	3.1×	3.0×10^{-73}
Nemotron Nano 30B	5.74	3.13	1.8×	2.4×10^{-21}
Qwen 3.5 122B	5.60	1.33	4.2×	6.4×10^{-88}

Result 2 — Calibration collapses: the wrong cells grow. Reduced search would be unsurprising if the model’s calibration *survived*: a leaner search budget could still be allocated to uncertain hops. It does not. Table 4 shows the six-category shift from benchmark to natural paraphrase for each model. The pattern is uniform across models: *Necessary search* collapses by 11–27 percentage points, and the lost mass moves predominantly into *Truly missed* (uncertain hop the model fails to search at all). *Par. redundant* also drops — which would be a positive in isolation — but the trade-off is unfavourable: the model loses correct searches faster than it loses wasted ones.

Table 4: Six-category profile shift under natural paraphrasing (Δ = Natural – Benchmark, in percentage points of 1800 hops). All three models lose *Necessary search* and gain *Truly missed*.

Model	Correct		Errors			
	Δ Nec. srch.	Δ Corr. skip	Δ Truly miss.	Δ X-hop cov.	Δ Seq. red.	Δ Par. red.
Gemini 3.1 Pro	−19.4	+14.3	+17.3	+2.0	−6.1	−8.2
Nemotron Nano 30B	−10.5	+3.4	+9.2	+1.4	−0.7	−2.7
Qwen 3.5 122B	−27.4	+15.5	+27.7	−0.3	−2.4	−13.1

Result 3 — The calibration gap persists or widens. Under natural paraphrasing the AUROC of entropy $\rightarrow$ search drops marginally for Gemini (0.557 $\rightarrow$ 0.528) and is essentially unchanged for the two open models. The headline finding of §5 — that entropy is a weak predictor of search — *persists, and degrades for the highest-volume searcher*. Surface form, not internal uncertainty, governs the policy.

Synthesis. The two results together rule out the most charitable interpretation of the volume drop. If the model had become “smarter” under natural framing — searching less but routing better — we would see PR cells shrink while *Necessary search* held steady. We instead see both cells move in the wrong direction together: the categories that grow are *Truly missed* and *Correct skip*, exactly what one would expect from a model that has shifted its overall search prior downward irrespective of per-hop uncertainty. Two complementary probes in the appendix support a specific mechanism: under natural framing, models commit to per-hop answers *earlier in the trace, before any search is issued* (Appendix C), and the growth of *Truly missed* is dominated by genuine misses rather than by downstream cascade failures from a wrong earlier hop (Appendix D).

7 Discussion

Why does framing flip behavior so cleanly? The probes in Appendix C are consistent with the natural-paraphrased question being interpreted as a casual, narrative request rather than a structured retrieval task. The model’s *search policy* then conditions on its read of the prompt’s register rather than on per-hop epistemic state. This is a behavioral analogue of the “context leads, parametric memory follows”

effect documented by Tao et al. (2024) for *generation*: here it is the *search-decision* policy, not the answer, that the model conditions on surface context.

Implications for tool-use training. The standard RAG / tool-use training paradigm — Toolformer-style supervised insertion (Schick et al., 2023), Self-RAG critique tokens (Asai et al., 2024), or distillation from agentic trajectories (Wang et al., 2025) — does not explicitly tie the decision to search to the model’s own uncertainty about the sub-step. Our finding suggests these methods may produce models whose search behavior generalizes poorly off the training distribution of question forms. A natural next step is training that explicitly conditions search on (a proxy for) per-step entropy, e.g. via on-policy SFT over uncertainty-labelled trajectories.

Limitations. (i) Per-hop attribution uses an LLM judge; we mitigate this with high-confidence-only ablations and substring-fallback checks, and the qualitative pattern is robust. (ii) 600 questions is large enough for cross-condition contrasts but constrains finer slicing (e.g. by hop-count $\times$ topic). (iii) Semantic entropy at $N = 5$ admits only 7 discrete values; finer probes (Kunitomo-Jacquin et al., 2026) may sharpen the calibration metrics. (iv) We do not vary the search backend; results may differ with stronger retrieval. (v) Only one paraphrase per question is evaluated; the generalization gap may be even larger across the full natural-language manifold.

8 Conclusion

Existing benchmarks tell us whether RAG helps; they do not tell us whether a model knows *when* to search. We introduce a protocol that does, grounded in per-hop semantic entropy and per-hop search attribution, and use it to profile three production-style LLMs through a six-category extended calibration view that distinguishes three correct decisions (*Necessary search*, *Correct skip*, *Cross-hop covered*) from three error modes (*Truly missed*, *Sequentially redundant*, *Parametrically redundant*). The profiles reveal qualitatively different failure styles across models; the AUROC of entropy $\rightarrow$ search confirms search is at best weakly aligned with the model’s own uncertainty; and a single cosmetic paraphrase — same information need, same gold answer — shifts every model toward the worst error cell, *Truly missed*. We argue this constitutes a hidden generalization gap in the search behavior of contemporary LLMs and that the protocol introduced here is a starting point for measuring and ultimately closing it.

References

- A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR*, 2024.
- S. Cheng et al. Understanding the interplay between parametric and contextual knowledge for large language models. *arXiv:2410.08414*, 2024.
- M. Farahani and R. Johansson. Deciphering the interplay of parametric and non-parametric memory in retrieval-augmented language models. *EMNLP*, 2024.
- S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.
- Y. Huang and J. X. Huang. A survey on retrieval-augmented text generation for large language models. *ACM Computing Surveys*, 2024.
- Y. Kim et al. Reliability across parametric and external knowledge: Understanding knowledge handling in LLMs. *arXiv:2502.13648*, 2025.
- L. Kunitomo-Jacquin et al. Evidential semantic entropy for LLM uncertainty quantification. *EACL*, 2026.
- J. Lee and J. Hockenmaier. Evaluating step-by-step reasoning traces: A survey. *arXiv:2502.12289*, 2025.
- Z. Li et al. Understanding and patching compositional reasoning in LLMs. *Findings of ACL*, 2024.
- S. Min et al. FActScore: Fine-grained atomic evaluation of factual precision in long-form text generation. *EMNLP*, 2023.

- K. Ning et al. WTU-Eval: A whether-or-not tool usage evaluation benchmark for large language models. *arXiv:2407.12823*, 2024.
- T. Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.
- Y. Tao et al. When context leads but parametric memory follows in large language models. *EMNLP*, 2024.
- H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *TACL*, 10:539–554, 2022.
- H. Wadhwa et al. From RAGs to rich parameters: Probing how language models utilize external knowledge over parametric information for factual queries. *WSDM*, 2026.
- Z. Wang et al. RARE: Retrieval-augmented reasoning modeling. *arXiv:2503.23513*, 2025.
- K. Wu, E. Wu, and J. Zou. ClashEval: Quantifying the tug-of-war between an LLM’s internal prior and external evidence. *NeurIPS*, 2024.
- J. Wu et al. MRKE: The multi-hop reasoning evaluation of LLMs by knowledge edition. *arXiv:2402.11924*, 2024.
- S. Yang et al. Do large language models latently perform multi-hop reasoning? *ACL*, 2024.
- H. Zhang et al. Evaluating the external and parametric knowledge fusion of large language models. *arXiv:2405.19010*, 2024.

A Extended Search-Calibration Plots for Nemotron and Qwen

Per-model extended-calibration plots for the two models not shown in §4. The bars use the same colour code as Figure 2: green / blue / purple on the left of the dashed line are correct decisions; orange / yellow / red on the right are the three distinct error categories.

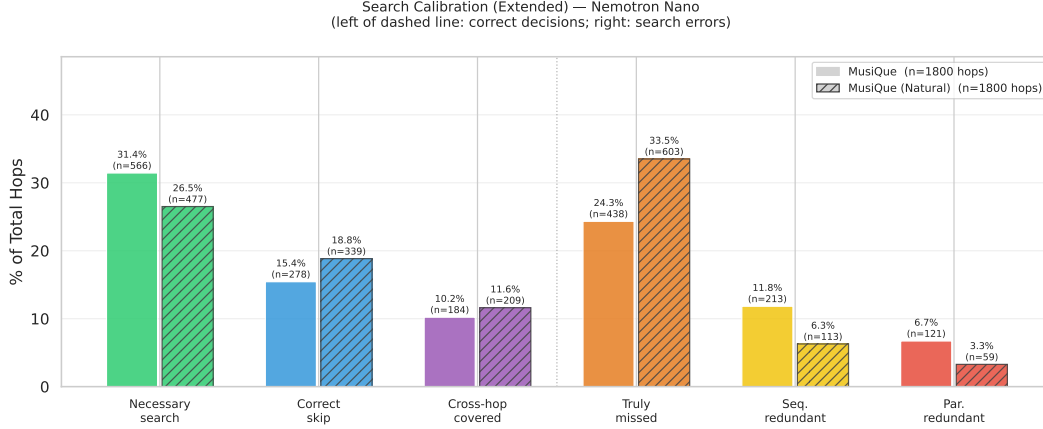
B Binary Search Behavior across Entropy Levels

A direct calibration view plots the search rate $P(\text{searched} \mid H = h)$ as a function of the discrete entropy level h produced by $N = 5$ closed-book sampling. Under well-calibrated behavior, this curve would rise monotonically from $H=0$ (the model should rarely search; it knows the answer) to $H=2.32$ (the model should almost always search; it has near-uniform disagreement across samples). What we observe instead (Figure 5) is a roughly flat curve for all three models: the search rate at $H=0$ is only a little below the rate at the highest entropy levels, and the intermediate levels are noisy. A χ^2 test of uniformity across entropy levels does reject the null for every model ($p < 10^{-4}$), so a small signal is statistically present — but the practical shape is binary: the model either searches at a roughly-constant rate or it doesn’t, with little graded sensitivity to how uncertain it is. This is the per-entropy view of the near-chance AUROC reported in §5.

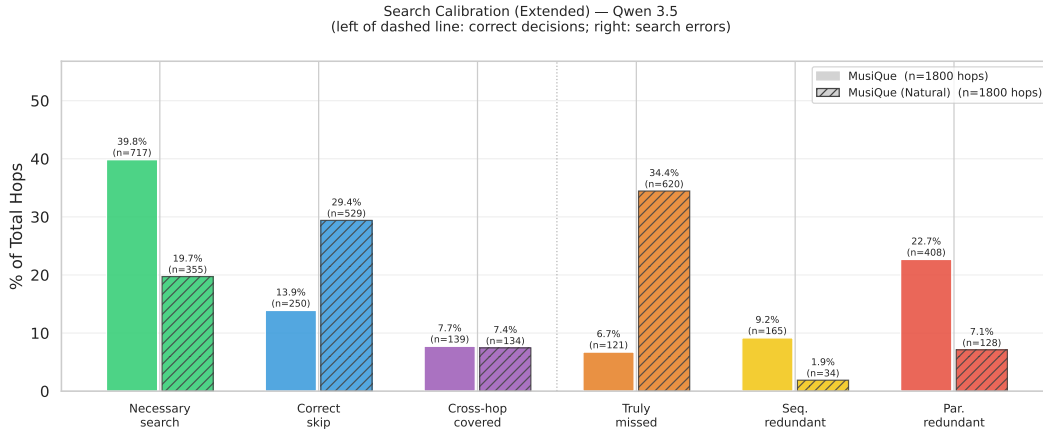
C Commitment-Locus Probe

Motivation. If natural framing causes models to skip searches they would otherwise have issued, *when* does the skip happen? One hypothesis is that the casual register of the paraphrased question invites a quick narrative answer, leading the model to commit to a per-hop answer early — before any search query is issued for that hop. The *commitment-locus probe* tests this directly.

Probe. For every (example, hop, variant) appearing in both the benchmark trace and the natural trace of the same model, an LLM judge (`gpt-oss:120b`) identifies the first turn at which the agent commits to a specific per-hop answer (asserts “X is Y” or treats Y as established and uses it downstream). Combined with the first turn at which a search query is attributed to that hop, this yields `committed_before_search` per (hop, variant). The decisive cell is `commit_no_search`: the agent committed to an answer for that hop and no search was ever attributed to it.



(a) Nemotron Nano 30B — coverage-failing under-searcher: *Truly missed* dominates (24.3% on the benchmark, 33.5% on natural), while both redundancy categories stay small.



(b) Qwen 3.5 122B — the most aggressive volume drop under natural paraphrasing (search rate 4.2× lower), with *Truly missed* expanding from 6.7% to 34.4%.

Figure 4: Extended search-calibration profiles for the remaining two models.

Result. Table 5 reports the headline rates; Figure 6 visualizes them. Across all three models, the rate of pre-search commitment rises substantially under natural framing. For Gemini, commitment-without-search jumps from 22.6% of hops on the benchmark to 51.9% on the natural rewrite. For Qwen the rise is 18.0% → 38.5%. Nemotron, which already searched the least on the benchmark, shows a smaller absolute increase.

Interpretation. The shift in commitment locus is a mechanistic explanation for the *Truly missed* growth reported in §6: under natural framing the model is not simply “forgetting” to search — it is *deciding* the answer earlier, often before any search would be triggered. This is consistent with the “context leads, parametric memory follows” phenomenon of Tao et al. (2024), re-cast at the level of the *search-decision* policy rather than the answer-generation policy.

D Missed-Hop Confounder Analysis

Motivation. *Truly missed* is the cell that grows the most under natural paraphrasing (Table 4). A potential confound is that some of these misses are not the model’s failure to act on its own uncertainty but downstream casualties of an earlier mistake: if hop k resolves to a wrong entity, the model has nothing to anchor a query for hop $k+1$. We separate the two.

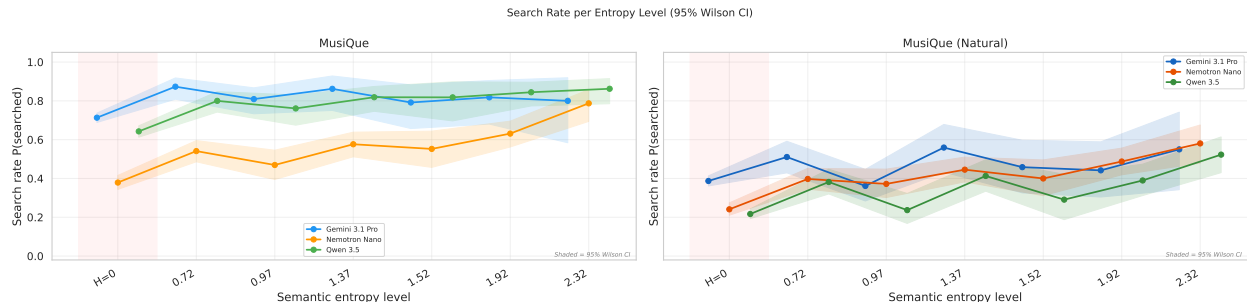


Figure 5: Search rate $P(\text{searched} \mid H = h)$ per entropy level, per model, on MuSiQue (left panel) and MuSiQue (Natural) (right panel). A well-calibrated policy would rise monotonically from $H=0$ to $H=2.32$. Observed curves are roughly flat on both datasets: behavior is approximately binary rather than graded with internal uncertainty. Shaded band = 95% Wilson CI.

Table 5: Commitment-locus rates per (model, variant). **commit_before_search**: agent committed before any search query attributed to the hop. **commit_no_search**: agent committed and no search ever attributed.

Model	Variant	Committed at all	Before search	No search at all
Gemini 3.1 Pro	Benchmark	78.5%	34.7%	22.6%
	Natural	56.1%	52.2%	51.9%
Nemotron Nano	Benchmark	52.2%	22.1%	21.5%
	Natural	54.0%	28.1%	27.8%
Qwen 3.5 122B	Benchmark	75.9%	19.6%	18.0%
	Natural	61.7%	38.9%	38.5%

Decomposition. For each hop in the *Truly missed* cell, we classify:

- **genuine_missed**: $\text{hop_index} = 0$, or every prior hop’s parametric runs were all correct. The model could have searched and chose not to.
- **cascade_possible**: $\text{hop_index} > 0$ and at least one prior hop had parametric accuracy < 1.0 . The model may have been “starved” of the anchor entity.

Result. Table 6 and Figure 8 show that the growth in *Truly missed* under natural paraphrasing is dominated by the *genuine_missed* component for all three models. Genuine misses roughly triple for Gemini ($84 \rightarrow 277$) and triple-plus for Qwen ($105 \rightarrow 369$); Nemotron rises by 42% ($240 \rightarrow 342$). This rules out the trivial explanation that the natural-paraphrase generalization gap is a downstream casualty of upstream error.

Table 6: *Truly missed* decomposition: genuine misses vs. cascade-possible misses, per (model, variant). Counts are of hops in the missed cell.

Model	Variant	Cascade possible	Genuine missed
Gemini 3.1 Pro	Benchmark	98	84
	Natural	254	277
Nemotron Nano	Benchmark	382	240
	Natural	470	342
Qwen 3.5 122B	Benchmark	155	105
	Natural	385	369

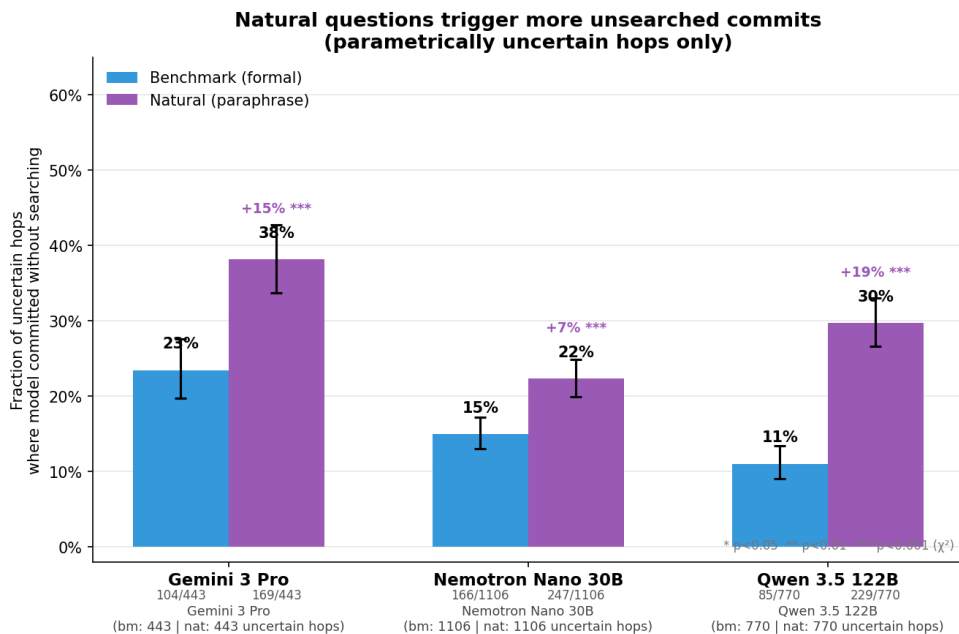


Figure 6: Pre-search commitment rate per (model, variant). Natural framing materially shifts the locus of the per-hop answer to *before* any search query is issued.

E Prompts

All LLM-judge calls in the protocol use the prompts reproduced below verbatim. Curly-brace placeholders are filled at runtime with the per-example fields named in the text.

E.1 Staleness Judge (Stage 0)

Model: gpt-oss:20b via Ollama.

System. You are an expert at identifying whether a factual question requires time-sensitive knowledge.

A question is STALE if its correct answer could plausibly change over time. This includes:

- Current officeholders (presidents, ministers, CEOs, etc.)
- Current statistics (population, GDP, rankings, prices)
- Current records (tallest building, richest person, etc.)
- Ongoing relationships (current spouse, current employer, current team)
- Any fact described as "current", "present", "now", "today", or "latest"

A question is NOT STALE if its correct answer is a fixed historical fact:

- Historical events and their dates
- Births, deaths, and biographical facts about historical figures
- Geographic facts (where a place is located, what country something is in)
- Authorship, invention, or creation of things
- Past achievements and awards
- Scientific or mathematical constants

For multi-hop questions, mark as STALE if ANY hop requires time-sensitive knowledge.

User. Classify the following question as STALE or NOT_STALE.

Question: {question}

Think through whether any part of answering this question requires knowing time-sensitive information (facts that could change over time, like current officeholders, current statistics, or ongoing relationships).

Respond in exactly this format:

Classification: STALE or NOT_STALE

Reason: <one sentence explaining why>

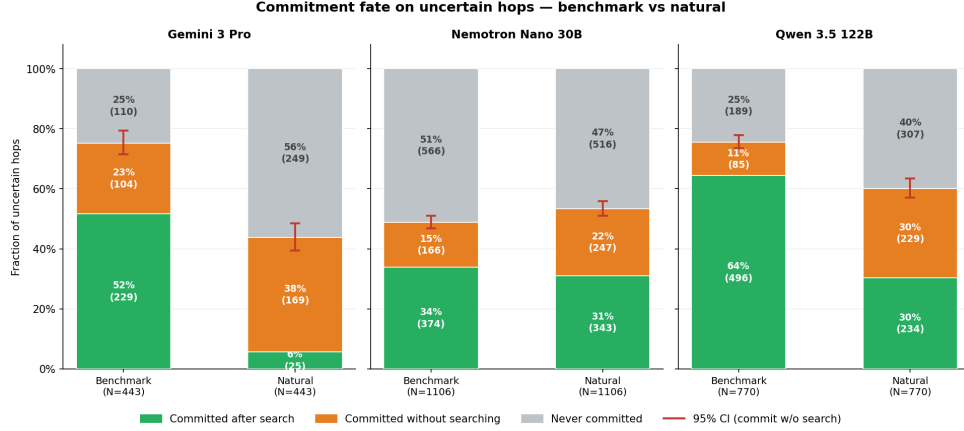


Figure 7: Commitment-fate breakdown per (model, variant). The growth of the orange “committed without searching” band under natural framing is the per-model picture of the generalization gap localized to a single behavioral mechanism.

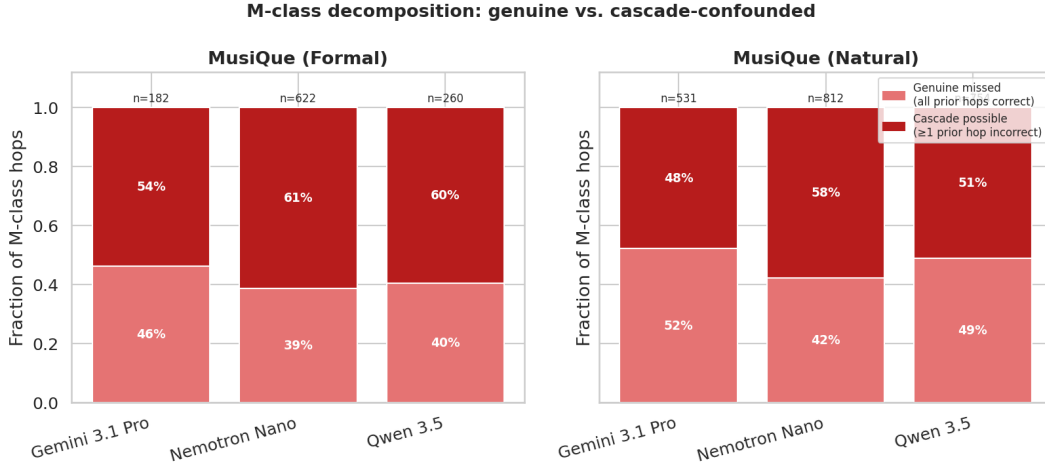


Figure 8: *Truly missed* cell decomposition. The dark (genuine_missed) component grows under natural framing for every model, ruling out cascade-driven explanations for the missed-cell expansion.

E.2 Semantic Clusterer (Stage 1)

Model: gpt-oss:120b via Ollama. The structured output is a list of integer cluster IDs of length $N = 5$.

Question: {question}

The following {n} answers were given independently to this question:
{answers}

Group them by semantic equivalence. Two answers belong to the same cluster if they express the same fact (minor wording differences are fine; "United States" and "USA" are the same). Use consecutive integer cluster IDs starting from 0.

E.3 Paraphrase Rewriter (Paraphrase generation)

Model: gpt-oss:20b via Ollama.

System. You are a prompt rewriter. You will be given a multi-hop benchmark question and the chain of sub-questions that reveal its structure. Your task is to rewrite the question so it sounds like a genuine, curious user asking -- not like a database traversal.

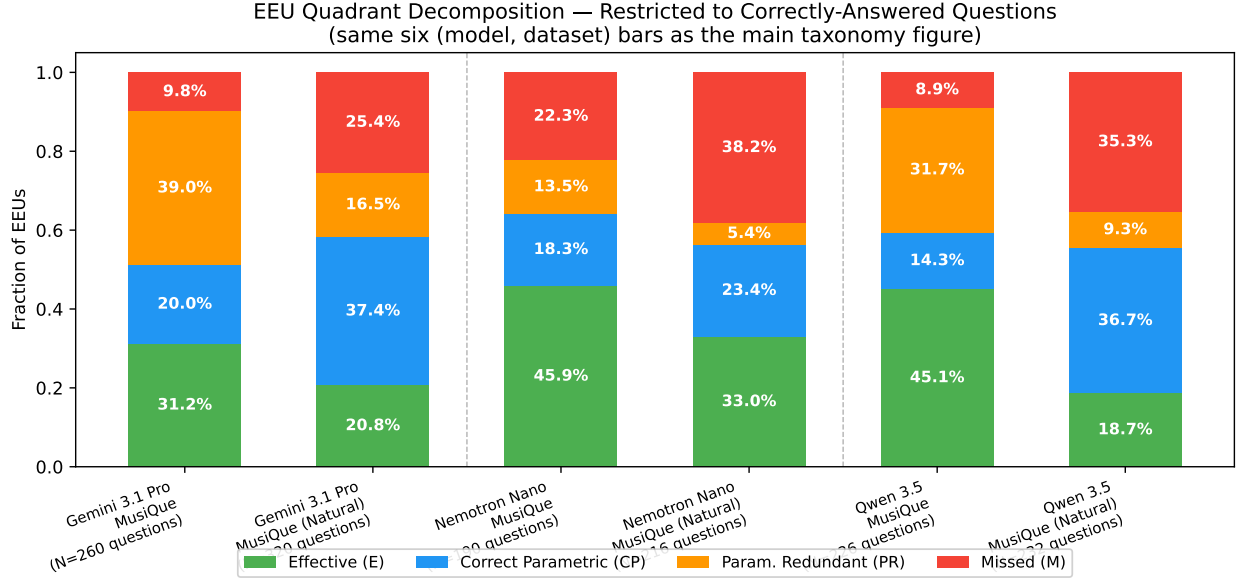


Figure 9: **Robustness check.** Reproduction of the headline taxonomy figure (Figure 1) restricted to questions on which the model produced the correct final aggregate answer. Even within this favourable sub-population, the natural-paraphrase bars show the same expansion of the red (*Missed*) segment and contraction of the green (*Effective*) segment. The calibration-and-paraphrase phenomena documented in the main text are therefore not artefacts of failure-mode questions: they appear inside the subset of questions the model actually got right.

Rules:

1. Do NOT mention any intermediate entity that appears only as a bridge in the hop chain. The user asking the question would not know that entity exists.
2. Do NOT use the "X of Y where Z did W" pattern. Avoid nested relative clauses that signal the hop structure.
3. Frame the question so the answer naturally warrants 2-4 explanatory sentences, not a single word or number. Prefer "why", "how", "tell me about", "explain", "what can you tell me about" phrasing where it fits naturally. If the answer is inherently factual (a location, a date), still frame it as "I'm curious about..." or "Could you tell me a bit about..." to invite context.
4. The rewritten question must preserve the same ultimate information need and have the same gold answer.
5. Return only the rewritten question text -- no explanation, no preamble, no quotes.

User. Original benchmark question: {question}

Hop chain (do NOT expose these steps in the rewrite):
{hops}

Gold answer: {answer}

Rewrite this as a natural user question following the rules:

E.4 Query→Hop Attribution Judge (Stage 3)

Model: gpt-oss:120b via Ollama. The structured output is the integer hop index plus a confidence label.

You are attributing a search query issued by an AI agent to one of the sub-questions (hops) that make up a multi-hop question.

Aggregate question: {aggregate_question}

Sub-questions (hops):
{sub_questions_text}

Agent's thinking before the search query:
{thinking}

Search query: {query}

Which sub-question (hop) is this search query primarily trying to answer?
Return the 0-based hop_index (0, 1, 2, ...) or -1 if the query is about the aggregate question or doesn't clearly target a specific sub-question.

E.5 Commitment-Locus Judge (Appendix C)

Model: gpt-oss:120b via Ollama.

You are analyzing a transcript of an AI agent solving a multi-hop question by interleaving reasoning, search-tool calls, and search results. Each turn in the transcript is prefixed with [TURN i: ROLE ...].

Aggregate question: {aggregate_question}

Target sub-question (one hop of the chain): {hop_question}

Gold answer for this sub-question: {gold_answer}

Your task: identify the FIRST turn at which the agent commits to a specific answer for the TARGET SUB-QUESTION above (not the aggregate question and not any other hop).

"Commits" means:

- Asserts a definite answer ("X is Y", "the answer is Z"), OR
- Treats an answer as established and reuses it in subsequent reasoning (e.g. forms the next hop's search query as if the prior answer is settled).

If a turn merely lists candidate answers, expresses doubt, or fetches information without concluding, it is NOT a commitment. The very first turn that satisfies the above is the commitment turn.

Notes:

- Turn indices are 0-based and match the [TURN i: ...] prefix.
- Return -1 for first_committed_turn_index if no commitment occurs anywhere in the transcript.
- matches_gold compares the COMMITTED answer to the GOLD answer; commitment exists even when wrong.

==== TRANSCRIPT ====

{transcript}

==== END TRANSCRIPT ====